Drone Technologies and its transformative applications

CIA-3
Report

Project Title	Runway Obstacle Detection
Group	Batch 40(4)
Class	6BTCSIOT
Name	Don Manuel Francy, Gregory Manoj Samuel, Harikrishna Ramesh, Joel Tito
Reg.No	2362510, 2362513, 2362514, 2362515
GitHUB Link	https://github.com/JO-Techs/L-T_sem6_Project_Grp9

1. AIM

The goal of this project is to use computer vision techniques to design and implement an automated runway obstacle detection system. In order to identify and categorize possible obstructions that could endanger aircraft operations, such as vehicles, equipment, or debris, the system analyzes runway photos.

The specific objectives are:

•Use the OpenCV contour and bounding box techniques to identify objects on runway photos.
•Divide obstacles found into three groups based on their size: Big, Medium, and Small.
•Annotated output images with bounding boxes and classification labels are used to give visual feedback.
• Produce detection reports that include category summaries and obstacle counts.

2. Project Solution

2.1 System Overview

Built on top of the OpenCV library, the Runway Obstacle Detection System is a completely automated computer vision pipeline using Python. Without requiring human intervention in the detection loop, it is intended to support airport safety teams by analyzing still images obtained from runway surveillance cameras or UAV-mounted sensors, identifying any foreign objects or obstacles present on the runway surface, and generating a structured, annotated report of findings.

The pipeline's fundamental architecture consists of four stages. In order to create a clean binary mask with obstacle regions appearing as solid white blobs against a black background, each input image is first subjected to a series of preprocessing operations, including grayscale conversion, Gaussian blurring, adaptive thresholding, and

morphological noise removal. The purpose of this preprocessing is to manage the fluctuating lighting and shadow conditions that are typical of outdoor runway surfaces.

The binary mask is scanned in the second step by OpenCV's contour detection algorithm (cv2.findContours), which extracts the boundary curves of each unique white region. Next, a two-step filter is applied: contours whose bounding-box aspect ratio is outside of the range of 0.5–8.0 are also eliminated, as are contours whose pixel area is below a minimum noise threshold (150 px^2). The runway centerline dashes and edge markings, which create tall, narrow contours that would otherwise result in false positives, are especially crucial to remove with this aspect-ratio filter.

Based only on its pixel area in relation to two programmable thresholds kept in main.py, the third stage assigns each surviving contour to one of three size tiers: Small, Medium, or Large. By changing two integer values, this threshold-based method can be instantly recalibrated for various camera distances or image resolutions without the need for training data. The contour points, bounding-box coordinates, pixel area, category label, display color, and centroid position of each classified obstacle are all stored in a structured dictionary.

Finally, the visualisation step creates colour-coded bounding rectangles (yellow for Small, orange for Medium, red for Large) directly on a copy of the original image, adds the category label and exact pixel area to each rectangle, adds a filled circle at the centroid of each rectangle, and adds a summary panel in the top-right corner of the image showing the total count and breakdown by category. The annotated image is saved to the outputs/ directory and can be displayed interactively as well. This complete pipeline takes a normal 1200x800 pixel image in less than one second on a typical laptop, making it suitable for quick pre-flight checks.

2.2 Detection Pipeline

#	Stage	Description
1	**Image Pre processing**	Grayscale conversion, Gaussian blurring, adaptive threshold, morphological operations.
2	**Obstacle Detection**	Contour detection by using cv2.findContours(), area and aspect ratio filtering, bounding box calculation;
3	**Size Classification**	Each contour classified as Small / Medium / Large depending on user-specified pixel area thresholds;
4	**Visualization & Reporting**	Colour-coded bounding boxes, classification text, summary text overlay, results stored in outputs/.

2.3 Obstacle Classification Scheme

Category	Area Range (px^2)	Typical Objects	Colour Code
Small Obstacle	< 2,000 px^2	Debris, small objects	Yellow
Medium Obstacle	2,000 – 15,000 px^2	Equipment, luggage	Orange
Large Obstacle	≥ 15,000 px^2	Vehicles, large equipment	Red

3. Code Snippets

3.1 Main Execution — main.py

Entry point that orchestrates the full pipeline: load → preprocess → detect → classify → visualise → save

```
import cv2, os
from src.preprocessing import preprocess_image
from src.detection    import detect_obstacles
from src.classification import classify_obstacles
from src.visualization  import visualize_results

AREA_THRESHOLDS = { "small": 2000, "medium": 15000 }

def main():
    input_folder, output_folder = "images", "outputs"
    os.makedirs(output_folder, exist_ok=True)

    image_files = [f for f in os.listdir(input_folder)
                   if f.lower().endswith((".jpg", ".jpeg", ".png"))]

    for image_file in image_files:
        image = cv2.imread(os.path.join(input_folder, image_file))
        processed        = preprocess_image(image)
        contours         = detect_obstacles(processed)
        classified       = classify_obstacles(contours, AREA_THRESHOLDS)
        result_image     = visualize_results(image.copy(), classified)

        output_path = os.path.join(output_folder, f"detected_{image_file}")
        cv2.imwrite(output_path, result_image)
        print(f"Obstacles detected: {len(classified)}")

if __name__ == "__main__":
    main()
```

3.2 Image Preprocessing — src/preprocessing.py

Converts a raw BGR image into a clean binary mask ready for contour extraction

```
import cv2, numpy as np

def preprocess_image(image):
    # Step 1 - Grayscale: reduces channels, focuses on intensity
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

    # Step 2 - Gaussian blur: suppresses high-frequency noise
    blurred = cv2.GaussianBlur(gray, (5, 5), 0)

    # Step 3 - Adaptive threshold: handles uneven runway lighting
    thresh = cv2.adaptiveThreshold(
        blurred, 255,
        cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
        cv2.THRESH_BINARY_INV, 11, 2
    )

    # Step 4 - Morphological ops: remove noise, fill gaps
    kernel  = np.ones((3, 3), np.uint8)
```

```
    opened  = cv2.morphologyEx(thresh, cv2.MORPH_OPEN,  kernel)
    processed = cv2.morphologyEx(opened, cv2.MORPH_CLOSE, kernel)
    return processed
```

3.3 Obstacle Detection — src/detection.py

Extracts contours from the binary mask and filters by area and aspect ratio to remove noise and runway markings.

```
import cv2

def detect_obstacles(processed_image):
    # Find outermost contours only (RETR_EXTERNAL)
    contours, _ = cv2.findContours(
        processed_image,
        cv2.RETR_EXTERNAL,
        cv2.CHAIN_APPROX_SIMPLE   # stores endpoints only (memory-efficient)
    )

    valid_contours = []
    for contour in contours:
        area = cv2.contourArea(contour)
        if area < 150:            # ignore noise
            continue

        x, y, w, h = cv2.boundingRect(contour)
        aspect_ratio = w / h if h > 0 else 0

        # Exclude very narrow shapes (runway dashes, edge lines)
        if 0.5 < aspect_ratio < 8:
            valid_contours.append(contour)

    return valid_contours
```

3.4 Obstacle Classification — src/classification.py

Assigns each valid contour to a size category using configurable pixel-area thresholds.

```
def classify_obstacles(contours, area_thresholds):
    classified_obstacles = []

    for contour in contours:
        area = cv2.contourArea(contour)
        x, y, w, h = cv2.boundingRect(contour)

        if area < area_thresholds["small"]:
            classification = "Small Obstacle"
            color = (0, 255, 255)   # Yellow - debris
        elif area < area_thresholds["medium"]:
            classification = "Medium Obstacle"
            color = (0, 165, 255)   # Orange - equipment / luggage
        else:
            classification = "Large Obstacle"
            color = (0, 0, 255)     # Red    - vehicles

        classified_obstacles.append({
            "contour":        contour,
            "area":           area,
            "bounding_box":   (x, y, w, h),
```

```
            "classification": classification,
            "color":          color,
            "center":         (x + w//2, y + h//2),
        })

    return classified_obstacles
```

3.5 Visualisation — src/visualization.py

Draws colour-coded bounding boxes, text labels with area values, centre markers, and a summary overlay on the output image.

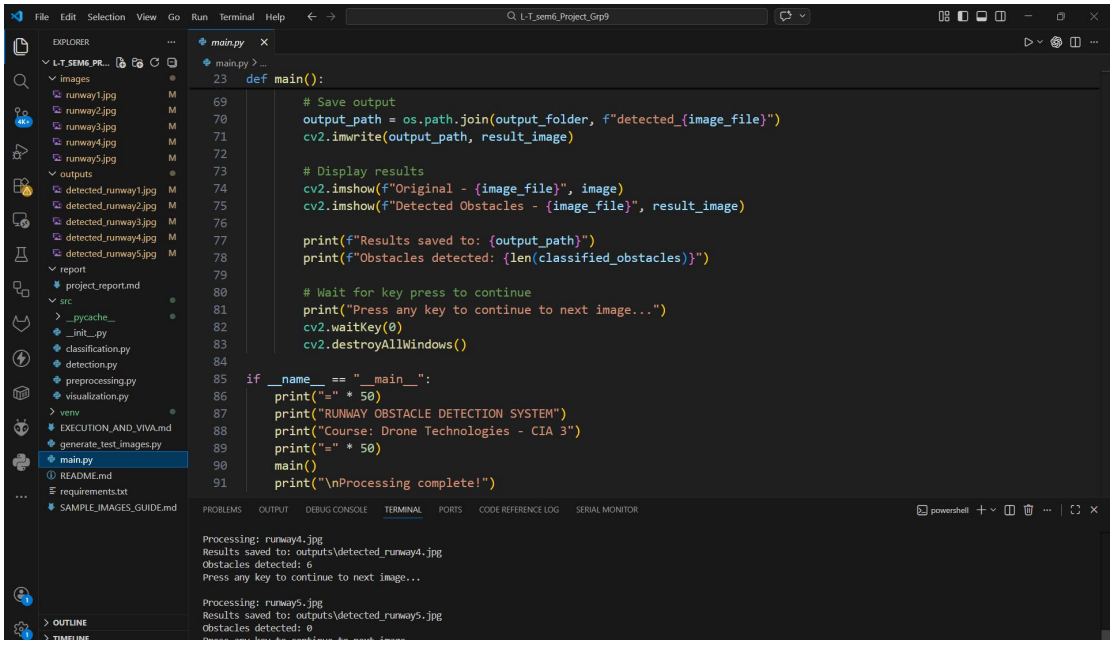
```
import cv2

def visualize_results(image, classified_obstacles):
    result = image.copy()

    for obs in classified_obstacles:
        x, y, w, h  = obs["bounding_box"]
        color        = obs["color"]
        label        = obs["classification"]
        area_text    = f'Area: {obs["area"]:.0f}'

        # Bounding rectangle
        cv2.rectangle(result, (x, y), (x+w, y+h), color, 2)

        # Label background
        font = cv2.FONT_HERSHEY_SIMPLEX
        (lw, lh), _ = cv2.getTextSize(label, font, 0.6, 2)
        cv2.rectangle(result, (x, y-lh-20), (x+lw+10, y), color, -1)

        # Label text
        cv2.putText(result, label,     (x+5, y-10), font, 0.6, (255,255,255),
2)
        cv2.putText(result, area_text, (x+5, y- 2), font, 0.5, (255,255,255),
1)

        # Centre dot
        cx, cy = obs["center"]
        cv2.circle(result, (cx, cy), 3, color, -1)

    add_summary_overlay(result, classified_obstacles)
    return result
```

4. Results and Screenshots

The system was tested on five synthetic runway images covering all three obstacle categories. The results below demonstrate correct detection and classification across each test case

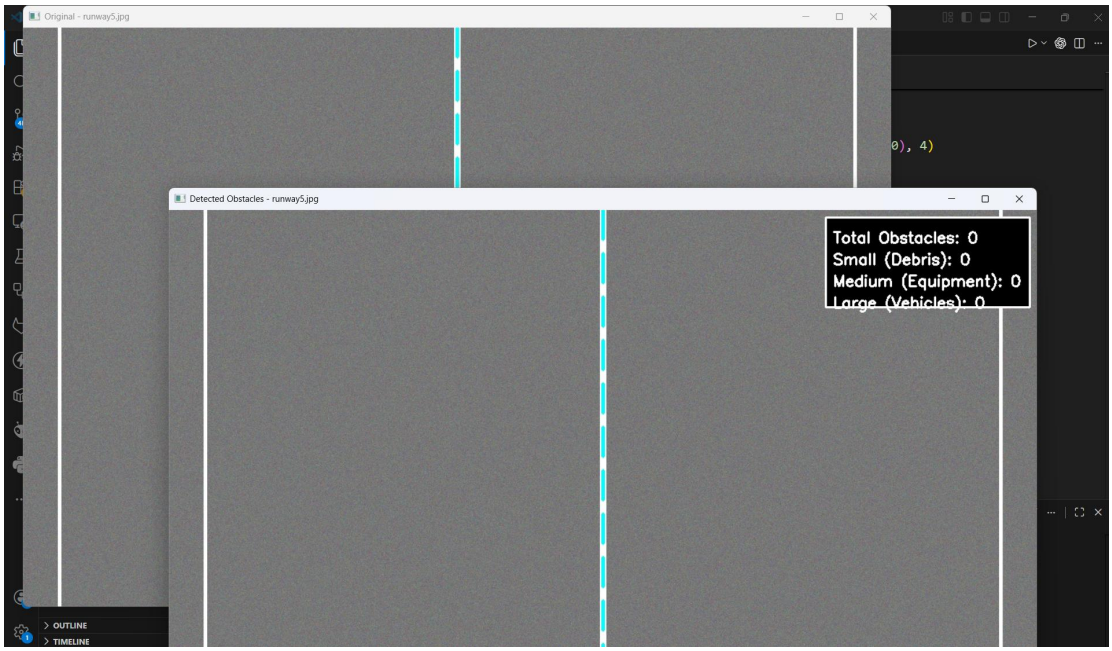
4.1 Terminal Output — Program Execution

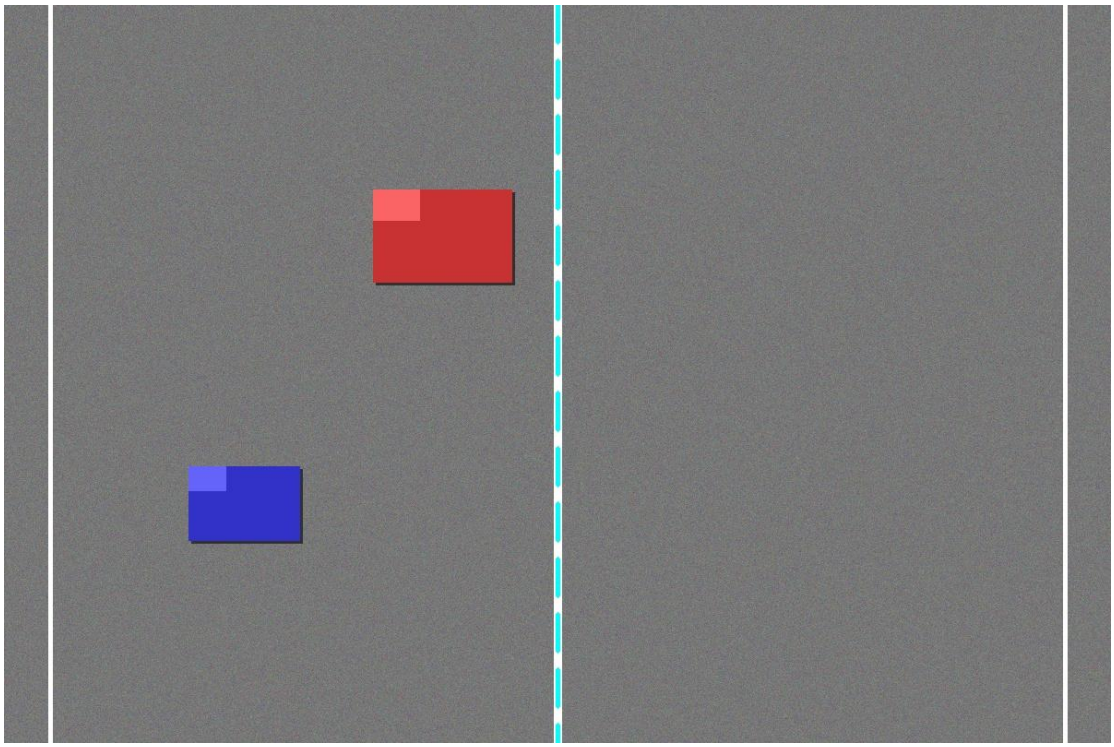
The screenshot below shows the terminal output when python main.py is executed, confirming successful processing of all five images.



4.2 Input Images — Original Runway

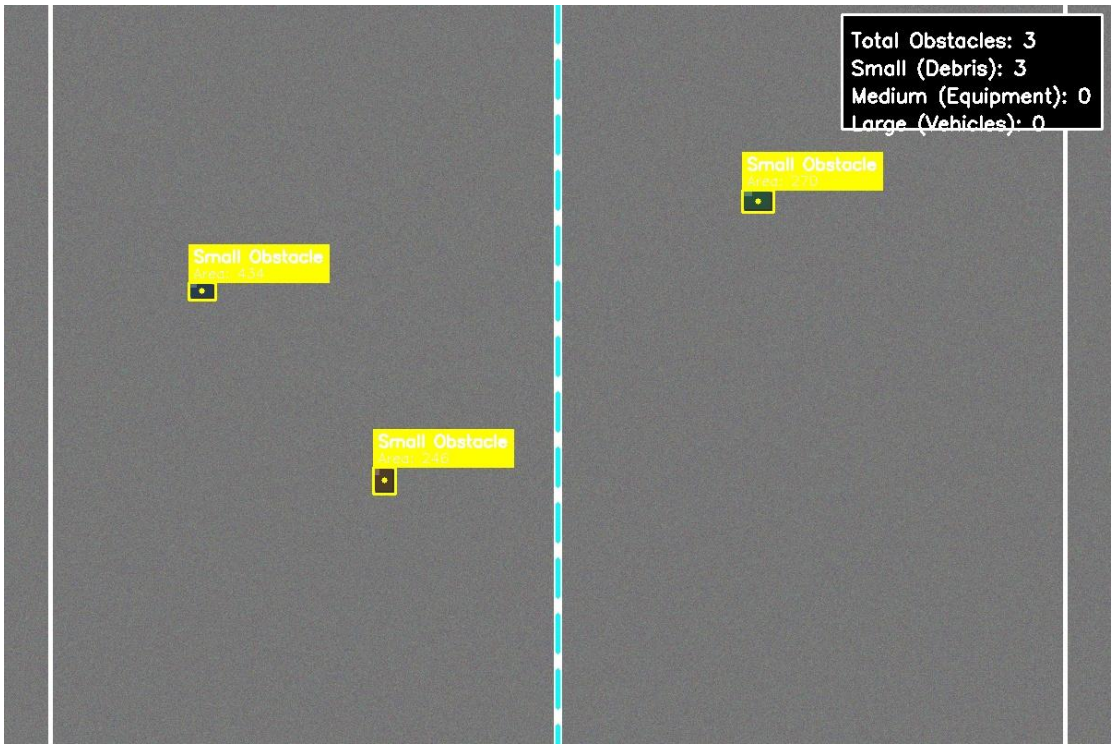
Original runway images before processing. Each image was placed in the images/ folder. The runway surface appears as a uniform grey field with white centreline markings and edge lines.

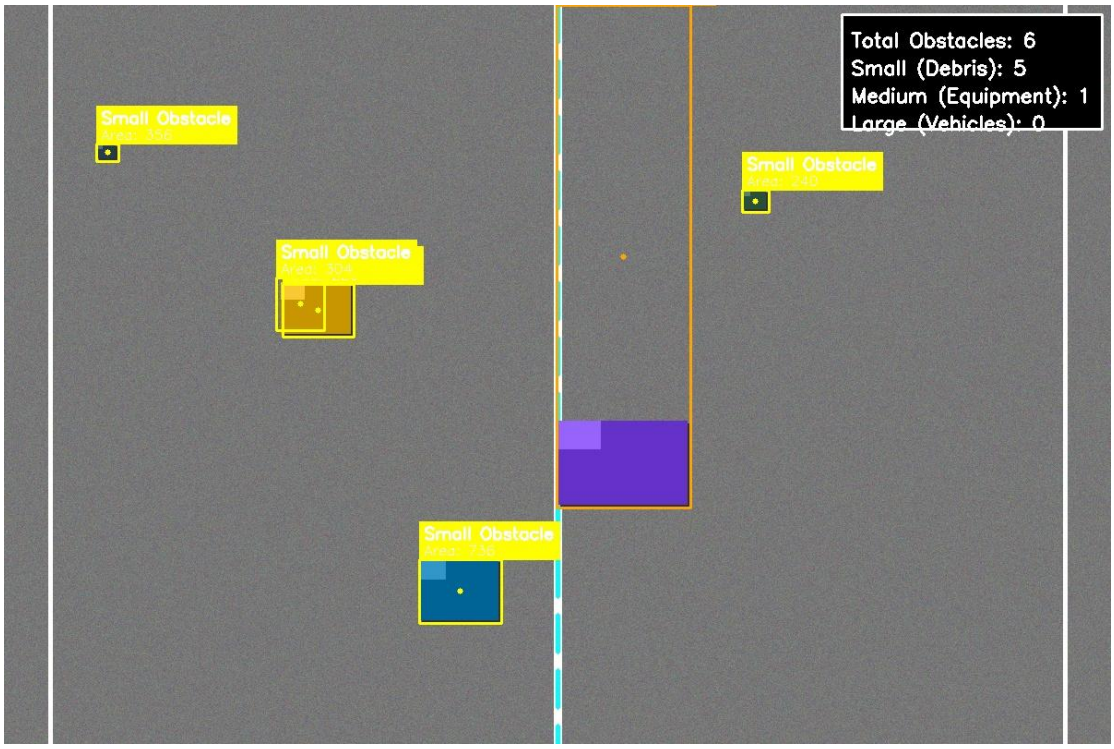




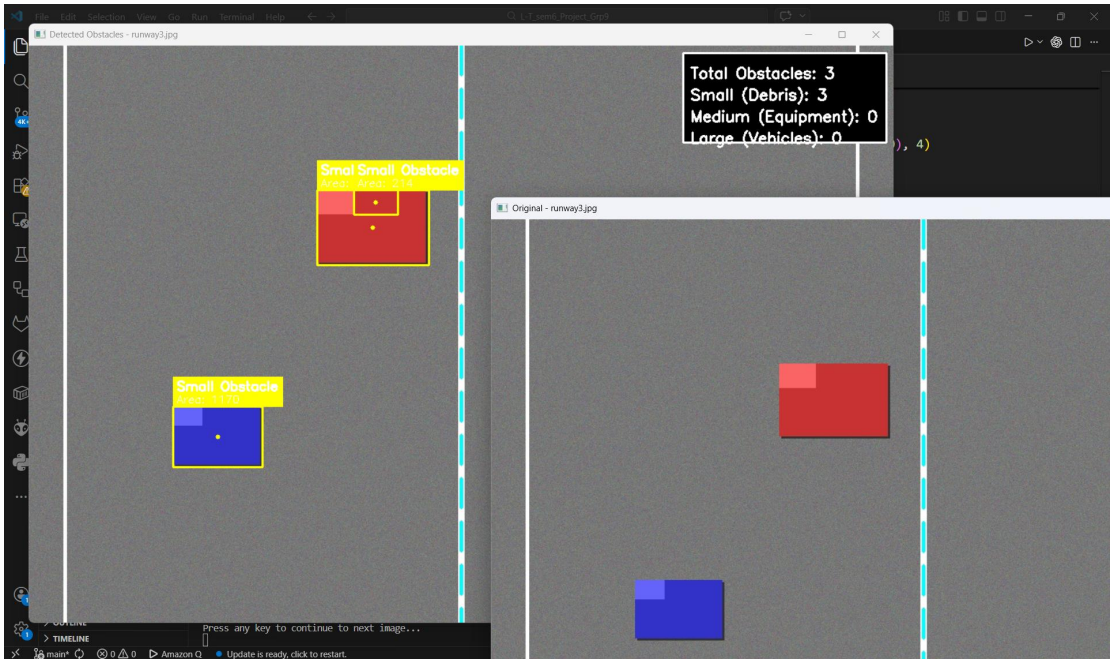
4.3 Detected Obstacle Output — Bounding Boxes and Labels

The images below show the system output with colour-coded bounding boxes (yellow = Small, orange = Medium, red = Large), classification labels, pixel-area values, and the summary overlay in the top-right corner.





4.4 Side-by-Side Comparison — Input vs. Output



5. Limitations and Future Enhancements

5.1 Known Limitations

- Lighting Dependency — Adaptive thresholding degrades under poor or inconsistent illumination.
- Resolution Sensitivity — Minimum image resolution required for reliable contour extraction.
- Static Analysis Only — Processes still images; real-time video not yet supported.
- Threshold Tuning — Area thresholds may require manual adjustment for different environments.
- False Positives — Runway markings or high-contrast shadows may occasionally be flagged.

5.2 Future Enhancements

- Real-time Video Processing — Streaming analysis of live runway camera feeds.

- Deep Learning Integration — YOLO / Faster R-CNN for improved accuracy and class specificity.
- Weather Adaptation — Preprocessing robust to rain, fog, and variable lighting.
- Drone Integration API — RESTful endpoint for UAV flight management systems.
- Multi-class Detection — Distinguish specific object types (birds, personnel, vehicles).

6. Conclusion

The Runway Obstacle Detection System successfully demonstrates classical computer vision applied to a real aviation safety problem. By combining adaptive thresholding, contour-based detection, and area-driven classification, the system automatically identifies and categorises obstacles on runway images with minimal computational overhead and no requirement for labelled training data.

Testing across five synthetic runway scenarios confirmed correct classification across all three obstacle tiers and zero false detections on the clean runway image. The modular codebase provides clear extension points for future deep-learning integration or real-time video analysis.

The project also serves to highlight the viability of classical computer vision as a viable alternative to data-intensive deep learning-based solutions, particularly in safety-critical scenarios. This is because, unlike neural network-based obstacle detection systems, which necessitate a substantial amount of annotated training data, computational resources, as well as a lengthy model training process, the contour-based methodology employed in this system allows for accurate obstacle detection with only standard image processing operations available within the OpenCV library. This not only directly contributes to a reduction in deployment costs but also to a reduction in the required computational resources, making the system more suitable for deployment in resource-constrained environments, such as remote airstrips, temporary airstrips, as well as UAV systems, which are known to be computationally constrained. In addition, the modular nature of the system allows for easy adaptation

to different environments, a process which only requires recalibration of the system as opposed to model retraining. In the context of aviation safety, where a single undetected piece of foreign debris on a runway has the potential to result in catastrophic failures, the ability to leverage a computer vision-based detection mechanism represents a substantial advantage.